ClawInc Student Setup Guide: Your Own AI Agent Company

MKT 518 — Deploying OpenClaw on DigitalOcean with Telegram Control

J. Christopher Westland

2026-04-01

Contents

What you will build: A company of 5 autonomous AI agents (Henry, Coder, Scout, Writer, Watcher) running 24/7 on a cloud server. You control them by sending text or voice messages in Telegram — from your phone or desktop — and they autonomously research, analyze, code, write reports, and monitor your system.

---

1 Part 1: Create Your DigitalOcean Droplet

1.1 Create a DigitalOcean Account

1. Go to digitalocean.com and sign up (use your university email).
2. Add a payment method. A droplet costs **$6/month** (the 1 GB RAM / 1 vCPU plan).
3. You can use a referral link to get $200 in free credits for 60 days.

1.2 Create a Droplet

1. Click **Create → Droplets** in the top menu.
2. Choose these settings:

Setting	Value
Region	Choose the closest to you (e.g., New York, San Francisco)
Image	Ubuntu 24.04 (LTS) x64
Size	Basic → Regular → **$6/mo** (1 GB RAM, 1 vCPU, 25 GB disk)
Authentication	Password — set a strong root password and save it
Hostname	`ClawInc` (or your company name)

3. Click **Create Droplet**. Wait 60 seconds for it to boot.
4. Note your droplet's **IP address** (shown in the DigitalOcean dashboard). You will use this everywhere.

1.3 Connect to Your Droplet

You need an SSH terminal. Options:

- **Windows:** Use PuTTY, or Windows Terminal (`ssh root@YOUR_IP`)
- **Mac/Linux:** Open Terminal and run `ssh root@YOUR_IP`
- **Browser:** In DigitalOcean, click your droplet → **Access** → **Launch Droplet Console**

```
ssh root@YOUR_DROPLET_IP
# Enter your root password when prompted
```

You should see a prompt like `root@ClawInc:~#`. You are now inside your server.

---

2 Part 2: Create Your Telegram Bots

You need **4 Telegram bots** (one per agent). This is free and takes about 5 minutes.

2.1 Install Telegram

- **Phone:** Download Telegram from the App Store or Google Play.
- **Desktop:** Download from telegram.org/apps.
- **Browser:** Go to web.telegram.org.

Create a Telegram account if you do not have one.

2.2 Create Bots with BotFather

1. In Telegram, search for **@BotFather** (the official Telegram bot creation service — blue checkmark).
2. Start a chat and send the command `/newbot`.
3. BotFather will ask for a name and a username. Create **4 bots** one at a time:

Bot Name	Bot Username (must end in `bot`)	Agent Role
Henry ClawBot	`henryclawbot` (or similar)	Chief of Staff — your main command interface
Coder ClawBot	`coderclawbot`	Software Engineer — writes and runs code
Scout ClawBot	`scoutclawbot`	Research Analyst — web research and trends
Writer ClawBot	`writerclawbot`	Content Creator — memos and reports

4. After each `/newbot`, BotFather gives you a **bot token** that looks like:

   `8732631641:AAHu1OuUh8uRXqpZqH_6G77DMOwIAXVaRKU`

5. **Save all 4 tokens in a text file.** You will need them in Part 4.

   Tip: To create the next bot, just send `/newbot` to BotFather again. You do not need to start a new conversation.

---

3 Part 3: Get an Anthropic API Key

Your agents run on Claude models. You need an Anthropic API key.

1. Go to console.anthropic.com and sign up.
2. Go to **API Keys** → **Create Key**.
3. Name it `ClawInc` and copy the key. It starts with `sk-ant-api03-...`
4. **Save it.** You cannot view it again after closing the page.

   Note: You will be charged per token used. For a class project, typical usage is $5–$20/month depending on how active your agents are.

---

4 Part 4: Deploy OpenClaw on Your Droplet

4.1 Upload the Deploy Package

From your **local computer** (not the server), run this command to copy the deploy files to your server. Replace `YOUR_IP` with your droplet's IP address:

```
scp -r deploy/ root@YOUR_IP:/root/deploy/
```

If you are on Windows and don't have `scp`, use WinSCP (free GUI tool): - Protocol: SCP - Host: YOUR_IP - Username: root - Password: your root password - Copy the entire `deploy/` folder to `/root/deploy/` on the server.

4.2 Edit the Deploy Script

Before running the script, you need to put in your own API keys. On your server:

```
nano /root/deploy/deploy-openclaw.sh
```

Find the **CONFIGURATION** section at the top and fill in your values:

```
# ============================================================
# CONFIGURATION - EDIT THESE
# ============================================================

ANTHROPIC_API_KEY="sk-ant-api03-YOUR-KEY-HERE"

TELEGRAM_TOKEN_HENRY="YOUR_HENRY_BOT_TOKEN"
TELEGRAM_TOKEN_CODER="YOUR_CODER_BOT_TOKEN"
TELEGRAM_TOKEN_SCOUT="YOUR_SCOUT_BOT_TOKEN"
TELEGRAM_TOKEN_WRITER="YOUR_WRITER_BOT_TOKEN"
```

Save the file: press `Ctrl+X`, then `Y`, then `Enter`.

4.3 Run the Deploy Script

```
chmod +x /root/deploy/deploy-openclaw.sh
/root/deploy/deploy-openclaw.sh
```

This script takes **5–10 minutes** and will:

- Create 2 GB of swap space (essential for a 1 GB RAM server)
- Install Node.js 24 and OpenClaw
- Set up the 5 agent workspaces

- Configure the firewall
- Attempt to start the gateway

Watch the output scroll by. At the end you should see green checkmarks.

4.4 Fix the Configuration (Required — Schema Update)

The deploy script uses an older config format. You need to replace `openclaw.json` with the corrected version that matches OpenClaw 2026.x. Run these commands on your server:

```
cat > /home/clawuser/.openclaw/openclaw.json << 'EOF'
{
  "env": {
    "ANTHROPIC_API_KEY": "YOUR_ANTHROPIC_API_KEY"
  },
  "gateway": {
    "mode": "local"
  },
  "agents": {
    "defaults": {
      "model": { "primary": "anthropic/claude-sonnet-4-5-20250929" }
    },
    "list": [
      {
        "id": "henry",
        "name": "Henry",
        "default": true,
        "workspace": "~/.openclaw/workspace-henry",
        "agentDir": "~/.openclaw/agents/henry",
        "model": "anthropic/claude-opus-4-6",
        "thinkingDefault": "high"
      },
      {
        "id": "coder",
        "name": "Coder",
        "workspace": "~/.openclaw/workspace-coder",
        "agentDir": "~/.openclaw/agents/coder",
        "model": "anthropic/claude-sonnet-4-5-20250929",
        "thinkingDefault": "high"
      },
      {
        "id": "scout",
        "name": "Scout",
        "workspace": "~/.openclaw/workspace-scout",
        "agentDir": "~/.openclaw/agents/scout",
        "model": "anthropic/claude-haiku-4-5-20251001",
        "thinkingDefault": "medium"
      },
      {
        "id": "writer",
        "name": "Writer",
        "workspace": "~/.openclaw/workspace-writer",
        "agentDir": "~/.openclaw/agents/writer",
        "model": "anthropic/claude-sonnet-4-5-20250929",
        "thinkingDefault": "medium"
```

```
    },
    {
      "id": "watcher",
      "name": "Watcher",
      "workspace": "~/.openclaw/workspace-watcher",
      "agentDir": "~/.openclaw/agents/watcher",
      "model": "anthropic/claude-haiku-4-5-20251001",
      "thinkingDefault": "low"
    }
  ]
},
"channels": {
  "telegram": {
    "enabled": true,
    "dmPolicy": "open",
    "allowFrom": ["*"],
    "defaultAccount": "henry-bot",
    "accounts": {
      "henry-bot": {
        "botToken": "YOUR_HENRY_BOT_TOKEN",
        "dmPolicy": "open",
        "allowFrom": ["*"]
      },
      "coder-bot": {
        "botToken": "YOUR_CODER_BOT_TOKEN",
        "dmPolicy": "open",
        "allowFrom": ["*"]
      },
      "scout-bot": {
        "botToken": "YOUR_SCOUT_BOT_TOKEN",
        "dmPolicy": "open",
        "allowFrom": ["*"]
      },
      "writer-bot": {
        "botToken": "YOUR_WRITER_BOT_TOKEN",
        "dmPolicy": "open",
        "allowFrom": ["*"]
      }
    }
  }
},
"bindings": [
  { "agentId": "henry", "match": { "channel": "telegram", "accountId": "henry-bot" } },
  { "agentId": "coder", "match": { "channel": "telegram", "accountId": "coder-bot" } },
  { "agentId": "scout", "match": { "channel": "telegram", "accountId": "scout-bot" } },
  { "agentId": "writer", "match": { "channel": "telegram", "accountId": "writer-bot" } }
],
"memory": {
  "backend": "qmd",
  "qmd": {
    "searchMode": "search"
  }
},
```

```
  "logging": {
    "level": "info",
    "file": "~/.openclaw/logs/openclaw.log"
  },
  "cron": {
    "enabled": true
  }
}
EOF
```

Replace the placeholder values with your actual keys before running: - `YOUR_ANTHROPIC_API_KEY` → your `sk-ant-api03-...` key - `YOUR_HENRY_BOT_TOKEN` → the token BotFather gave you for the Henry bot - `YOUR_CODER_BOT_TOKEN` → the Coder bot token - `YOUR_SCOUT_BOT_TOKEN` → the Scout bot token - `YOUR_WRITER_BOT_TOKEN` → the Writer bot token

Then set permissions:

```
chmod 600 /home/clawuser/.openclaw/openclaw.json
chown clawuser:clawuser /home/clawuser/.openclaw/openclaw.json
```

Validate the config:

```
su - clawuser -c "openclaw config validate"
# Should print: Config valid: ~/.openclaw/openclaw.json
```

4.5 Fix the Systemd Service

The deploy script starts the gateway with the wrong command. Fix it:

```
# Fix the command (gateway run, not gateway start)
sed -i 's|ExecStart=/usr/bin/openclaw gateway start|ExecStart=/usr/bin/openclaw gateway run|' \
    /etc/systemd/system/openclaw.service

# Fix the PrivateTmp setting that blocks the tmp directory
sed -i 's|PrivateTmp=true|PrivateTmp=false|' /etc/systemd/system/openclaw.service

# Add performance environment variables
sed -i '/Environment=NODE_ENV=production/a Environment=OPENCLAW_NO_RESPAWN=1' \
    /etc/systemd/system/openclaw.service

# Create the required temp directory
CLAW_UID=$(id -u clawuser)
mkdir -p /tmp/openclaw-${CLAW_UID}
chown clawuser:clawuser /tmp/openclaw-${CLAW_UID}

# Reload and start
systemctl daemon-reload
systemctl restart openclaw
```

Wait 10 seconds, then check the service is running:

```
systemctl is-active openclaw
# Should print: active
```

4.6 Add Cron Jobs

Run these commands to set up the automated daily schedule:

```
# Wait for the gateway to be ready, then add cron jobs as clawuser
su - clawuser << 'CRONEOF'

openclaw cron add \
  --name "morning-research" --agent "scout" \
  --cron "0 8 * * *" --session isolated \
  --message "Run your morning research routine. Search the web for the latest trending topics in AI, ma

openclaw cron add \
  --name "daily-memo" --agent "writer" \
  --cron "0 9 * * *" --session isolated \
  --message "Compile the morning memo. Search Scout memory for today research briefing. Synthesize into

openclaw cron add \
  --name "overnight-worker" --agent "coder" \
  --cron "0 2 * * *" --session isolated \
  --message "Check your task queue and Henry recent delegations. Work on the highest-priority pending d

openclaw cron add \
  --name "health-check" --agent "watcher" \
  --cron "*/30 * * * *" --session isolated \
  --message "Check system resources (CPU, RAM, disk, swap), verify agents are responsive, review error l

openclaw cron add \
  --name "nightly-rnd" --agent "henry" \
  --cron "0 23 * * *" --session isolated \
  --message "Initiate the nightly R&D session. Review today memo from Writer, research from Scout, and a

openclaw cron add \
  --name "session-cleanup" --agent "watcher" \
  --cron "0 4 * * 0" --session isolated \
  --message "Perform weekly session cleanup. Archive sessions older than 7 days. Clean up temporary file

openclaw cron list
CRONEOF
```

You should see a table of 6 scheduled jobs.

---

5 Part 5: Verify Everything is Working

5.1 Check System Status

On your server:

```
su - clawuser -c "openclaw status"
```

Look for: - **Gateway:** should say `reachable` - **Agents:** should say `5` - **Channels:** Telegram bots should be listed

```
su - clawuser -c "openclaw agents list --bindings"
```

This shows all 5 agents and their Telegram bot connections.

5.2 Test Your Bots in Telegram

1. Open Telegram and search for your Henry bot by username (e.g., `@henryclawbot`).
2. Press **START** or send `/start`.
3. Send a message: `Hello! Who are you and what can you do?`
4. The Henry agent (powered by Claude Opus) should respond within a few seconds.

Do the same for Coder, Scout, and Writer bots.

> **If the bot does not respond:** Check the logs on your server with `journalctl -u openclaw -f` and look for error messages.

---

6 Part 6: Access the Dashboard in a Browser

OpenClaw has a built-in web dashboard at port 18789 on your server. Since your server's firewall blocks direct web access (for security), you access it through an **SSH tunnel** — a secure encrypted connection that forwards a local port on your computer to the server.

6.1 Open the SSH Tunnel

Mac/Linux — Terminal:

```
ssh -L 18789:localhost:18789 root@YOUR_DROPLET_IP
# Keep this terminal window open
```

Windows — PowerShell or Command Prompt:

```
ssh -L 18789:localhost:18789 root@YOUR_DROPLET_IP
```

Windows — PuTTY (GUI): 1. Open PuTTY 2. Enter your IP in **Host Name** 3. Go to **Connection → SSH → Tunnels** 4. Source port: `18789`, Destination: `localhost:18789`, click **Add** 5. Click **Open** and log in

6.2 Open the Dashboard

Once the tunnel is running, open your browser and go to:

`http://localhost:18789`

You will see the **OpenClaw Control Panel** showing:

- Live gateway status
- All 5 agents
- Active sessions
- Cron job schedule
- Memory contents
- Real-time logs

> **Keep the SSH tunnel terminal open** while using the dashboard. The tunnel closes when you close that window.

6.3 Dashboard Features

Section	What it shows
Overview	Server health, connected agents, memory stats
Agents	Each agent's model, workspace, routing rules
Sessions	Active and recent conversation sessions

Section	What it shows
Cron	Scheduled jobs, run history, next run times
Memory	Stored memories searchable across agents
Config	Edit openclaw.json directly in the browser
Logs	Real-time log stream from the gateway

---

7 Part 7: Customizing Your Agents

7.1 Editing an Agent's Personality (SOUL.md)

Each agent has a `SOUL.md` file that defines its system prompt — its personality, role, and behavior. To customize Henry's personality:

```
nano /home/clawuser/.openclaw/workspace-henry/SOUL.md
```

You can change the name, role, communication style, areas of expertise, etc. The file is plain Markdown. After saving, restart the gateway:

```
systemctl restart openclaw
```

7.2 Giving Agents Skills

Skills are pre-built task templates stored in each agent's `skills/` folder. To add a new skill to Scout:

```
ls /home/clawuser/.openclaw/workspace-scout/skills/
```

A skill file is a Markdown file with a `/skill-name` trigger. You can create custom skills — for example, a `market-analysis` skill that always follows a specific analytical framework.

7.3 Changing Agent Models

To upgrade Henry to use a different model, edit `openclaw.json`:

```
nano /home/clawuser/.openclaw/openclaw.json
```

Change the `model` field for any agent. Valid model IDs:

Model	Speed	Intelligence	Cost
`anthropic/claude-opus-4-6`	Slow	Highest	High
`anthropic/claude-sonnet-4-5-20250929`	Medium	High	Medium
`anthropic/claude-haiku-4-5-20251001`	Fast	Good	Low

After editing, validate and restart:

```
su - clawuser -c "openclaw config validate"
systemctl restart openclaw
```

---

8 Part 8: Using Your Agents for the Class Project

8.1 Talking to Your Agents

All commands start with a message to a Telegram bot. Be direct and specific. Examples:

To Henry (Chief of Staff):

```
Research the top 3 trends in social media marketing in Q1 2026,
then have Scout do a deep web analysis and Writer produce
a 2-page executive brief. Send it to me when done.
```

To Scout (Research):

```
Search the web for recent academic papers on customer segmentation
using AI. Summarize the top 5 findings with citations.
```

To Coder (Analysis):

```
I'm uploading a CSV of customer purchase data. Run a regression
analysis to find the top predictors of purchase frequency.
Generate a Python script and show me the output.
```

To Writer (Reports):

```
Write a 500-word executive summary of our Q1 marketing strategy,
using a formal business style. Include an executive abstract,
3 key findings, and 2 recommendations.
```

8.2 Voice Commands

Telegram supports voice notes. Record a voice note and send it to any bot — OpenClaw transcribes it and responds. This is useful for quick commands on mobile.

8.3 Checking Agent Memory

Agents remember previous conversations. To search what Scout has learned:

```
su - clawuser -c "openclaw memory search 'marketing trends' --agent scout"
```

8.4 Viewing Logs

Watch what your agents are doing in real time:

```
journalctl -u openclaw -f
```

Or in the browser dashboard under **Logs**.

---

9 Part 9: Useful Commands Reference

9.1 Server Management

```
# Check if gateway is running
systemctl is-active openclaw

# Restart the gateway (applies config changes)
systemctl restart openclaw
```

```
# View live logs
journalctl -u openclaw -f

# Check server resources
free -h           # RAM and swap
df -h /           # Disk space
top               # CPU and memory usage
```

9.2 OpenClaw Commands (run as clawuser)

```
# Switch to clawuser
su - clawuser

# Check overall status
openclaw status

# List agents with their Telegram bindings
openclaw agents list --bindings

# List cron jobs
openclaw cron list

# Run a cron job right now (for testing)
openclaw cron run morning-research

# Search agent memory
openclaw memory search "query" --agent scout

# Check Telegram bot connection
openclaw channels status --probe

# Run diagnostics
openclaw doctor

# Validate config
openclaw config validate
```

9.3 Editing Config

```
# Edit config directly
nano /home/clawuser/.openclaw/openclaw.json

# Validate after editing
su - clawuser -c "openclaw config validate"

# Restart to apply changes
systemctl restart openclaw
```

---

10 Part 10: Troubleshooting

10.1 Bot Not Responding

1. Check the service is running: `systemctl is-active openclaw`
2. Check logs: `journalctl -u openclaw -n 50 --no-pager`
3. Verify your bot token in `openclaw.json` matches what BotFather gave you
4. Run: `su - clawuser -c "openclaw channels status --probe"`

Common fixes: - Restart the gateway: `systemctl restart openclaw` - Re-validate your config: `su - clawuser -c "openclaw config validate"`

10.2 Config Invalid

```
su - clawuser -c "openclaw config validate"
```

If it shows errors, check: - JSON syntax (no trailing commas, all brackets matched) - `dmPolicy: "open"` requires `allowFrom: ["*"]` - All bot tokens are in the correct format (`NUMBER:LETTERS`)

10.3 Gateway Not Starting

```
# Check detailed error
journalctl -u openclaw -n 30 --no-pager

# Check if port is in use
ss -tlnp | grep 18789

# Run gateway manually to see error in real time
su - clawuser -c "openclaw gateway run"
```

10.4 Ran Out of Memory / Server Slow

```
# Check swap is active (should show 2GB)
free -h

# If swap missing, recreate it:
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
```

10.5 Lost Your Root Password

Use the DigitalOcean console (browser-based access) to reset your password: DigitalOcean Dashboard → Your Droplet → **Access → Reset Root Password**

---

11 Appendix: Quick-Start Checklist

Use this checklist to track your setup progress:

- ☐ Created DigitalOcean account and droplet (Ubuntu 24.04, $6/mo)
- ☐ Noted my droplet IP address: _____

☐ Installed Telegram on phone and/or desktop
☐ Created 4 bots via @BotFather and saved all 4 tokens
☐ Got Anthropic API key from console.anthropic.com
☐ Uploaded `deploy/` folder to `/root/deploy/` on server
☐ Edited deploy script with my API keys and bot tokens
☐ Ran deploy script successfully
☐ Replaced `openclaw.json` with corrected 2026.x schema version
☐ Fixed systemd service (`gateway run` instead of `gateway start`)
☐ Validated config with `openclaw config validate` — shows **Config valid**
☐ Service is active: `systemctl is-active openclaw` shows **active**
☐ Henry bot responds in Telegram
☐ Coder, Scout, Writer bots all respond
☐ Dashboard accessible at http://localhost:18789 (via SSH tunnel)
☐ All 6 cron jobs added and visible in `openclaw cron list`

---

12 Appendix: Agent Roster Quick Reference

Agent	Telegram Bot	Model	Best For
Henry	@YourHenryBot	Claude Opus 4.6	Orchestrating tasks, strategy, delegating to other agents
Coder	@YourCoderBot	Claude Sonnet 4.5	Writing Python/R code, data analysis, debugging
Scout	@YourScoutBot	Claude Haiku 4.5	Web research, trend monitoring, news summaries
Writer	@YourWriterBot	Claude Sonnet 4.5	Reports, memos, executive summaries, content
Watcher	*(no bot)*	Claude Haiku 4.5	Automated system health monitoring (background only)

---

13 Appendix: Cron Schedule Reference

Time (CST)	Agent	Task
Every 30 min	Watcher	System health check
8:00 AM daily	Scout	Web research scan for AI/marketing trends
9:00 AM daily	Writer	Compile morning intelligence memo
11:00 PM daily	Henry	Nightly R&D strategy session
2:00 AM daily	Coder	Work on queued development tasks
4:00 AM Sundays	Watcher	Weekly session cleanup and archiving

For questions about this setup, contact your instructor. For OpenClaw documentation, see the built-in docs at `http://localhost:18789` *(requires SSH tunnel).*